

National Supercomputing Mission



Centre for HPC Upskilling & Knowledge-sharing



Centre for Development of Advanced Computing

Introduction to
CUDA

SSDG C-DAC BENGALURU

CONTENT

Introduction to CUDA

NVIDIA GPGPU Architecture

CUDA Environment Setup

CUDA Programming

Conclusion

References

SSDG C-DAC BENGALURU

Introduction to CUDA

- HPP Review
- CUDA Evolution
- CUDA Basics
- Benchmarking

SSDG C-DAC BENGALURU

Introduction to CUDA : HPP Review

- **Serial**: tasks run sequentially. (task starting only after the previous one has completed)

- **Concurrent**: tasks run on the same core in time sharing.

- **Simultaneous / Parallel**: tasks run at the same time, on different cores.

SSDG C-DAC BENGALURU

Introduction to CUDA : HPP Review

Multi-processor Programming:

"A parallel programming model where all processing units are of the same type."

- Examples: OpenMP on a multi-core CPU (used before OpenGL/DirectX and CUDA)

SSDG C-DAC BENGALURU

Introduction to CUDA : HPP Review

Heterogeneous Parallel Programming:

"A parallel programming model where different types of processors work together to execute tasks."

- Examples:

  CUDA: CPU + GPU

  OpenCL/SYCL with multiple device types.

SSDG C-DAC BENGALURU

Introduction to CUDA : HPP Review

- **CPU**: ALU + CU
- **GPU**: ALU + CU (Light)
- **GPGPU**: General-Purpose computing on Graphics Processing Units.

SSDG C-DAC BENGALURU

Introduction to CUDA : HPP Review

- **TPU**: ALU + CU (Minimal)

- **XPU**: X-Processing Unit CPU|GPU|TPU

- **Accelerator**: A specialized coprocessor designed to offload and speed up particular workloads by providing hardware tailored to those tasks.

SSDG CDAC BENGALURU

Introduction to CUDA : HPP Review

- CPU clock speed improvements stalled due to physical and thermal limits.

- Multicore CPUs emerged but offer limited parallelism.

- GPUs evolved from fixed-function graphics processors to programmable, massively parallel processors.

SSDG C-DAC BENGALURU

Introduction to CUDA : CUDA Evolution

- CUDA was introduced in 2006 with NVIDIA's GeForce 8800 GTX.
- Before CUDA, GPU programming was complex and required disguising computations as graphics tasks via **OpenGL** or **DirectX**.
- CUDA provides direct, general-purpose access to GPU memory, eliminating the need for graphics APIs.

SSDG C-DAC BENGALURU

Introduction to CUDA : CUDA Evolution

- Limitation of early GPU computing before CUDA was lack of hardware floating-point units.

- CUDA supports IEEE-compliant floating-point arithmetic and arbitrary memory access.

SSDG C-DAC BENGALURU

Introduction to CUDA : CUDA Basics

- CUDA stands for Compute Unified Device Architecture.

- CUDA was introduced in 2006 with NVIDIA's GeForce 8800 GTX.

- CUDA enables general-purpose GPU computing using CUDA C/C++, an extension of standard C/C++ have extension (.cu).

SSDG, C-DAC BENGALURU

Introduction to CUDA : CUDA Basics

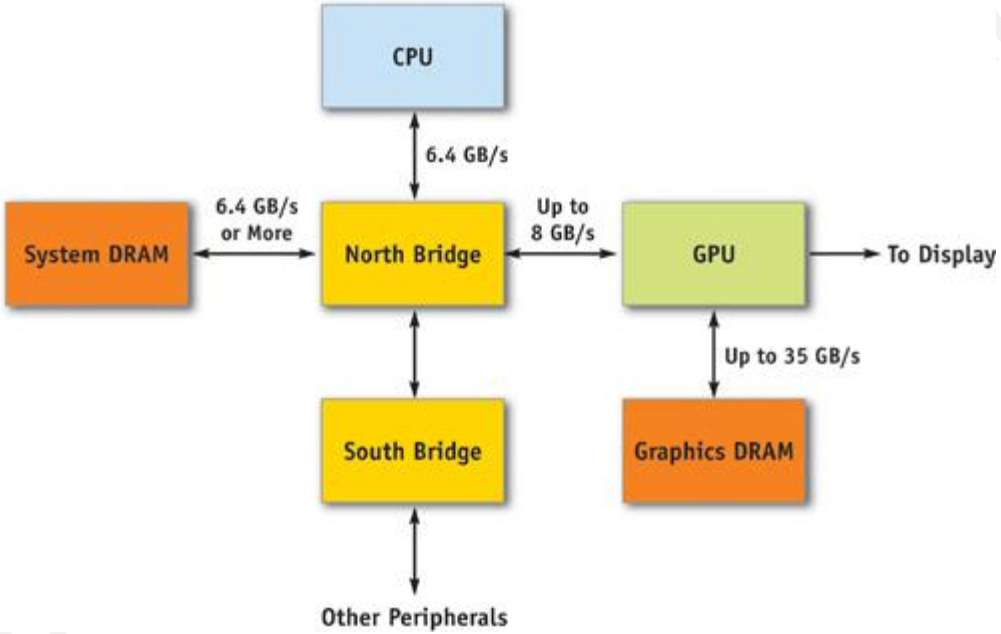


Figure: Computer Architecture having CPU and GPU
(Here NVIDIA GeForce 6)

Introduction to CUDA : CUDA Basics

- CUDA can be learnt in the following perspectives:

1. CUDA Application Constructs : PyCUDA
2. CUDA Runtime Environment Functions
3. CUDA Device Driver Development
4. CUDA Core Design (Hardware)

SSDG C-DAC BENGALURU

Introduction to CUDA : Benchmarking

- Use Case: Matrix Multiplication

1. Increase Matrix Size
2. Check “Gold Performance”

   (Performance on CPUs Only)
3. Check GPU Performance
4. Repeat from 1



SSDG C-DAC BENGALURU

NVIDIA GPGPU Architecture

- Host and Device Model
- NVIDIA GPGPU Architecture

SSDG C-DAC BENGALURU

NVIDIA GPGPU Architecture

Host: It is the part of the system that controls and manages execution.

In CUDA : Host = CPU

Device: It is a co-processor or accelerator that executes compute-intensive portions of the code.

In CUDA : Device = GPU

SSDG C-DAC BENGALURU

NVIDIA GPGPU Architecture

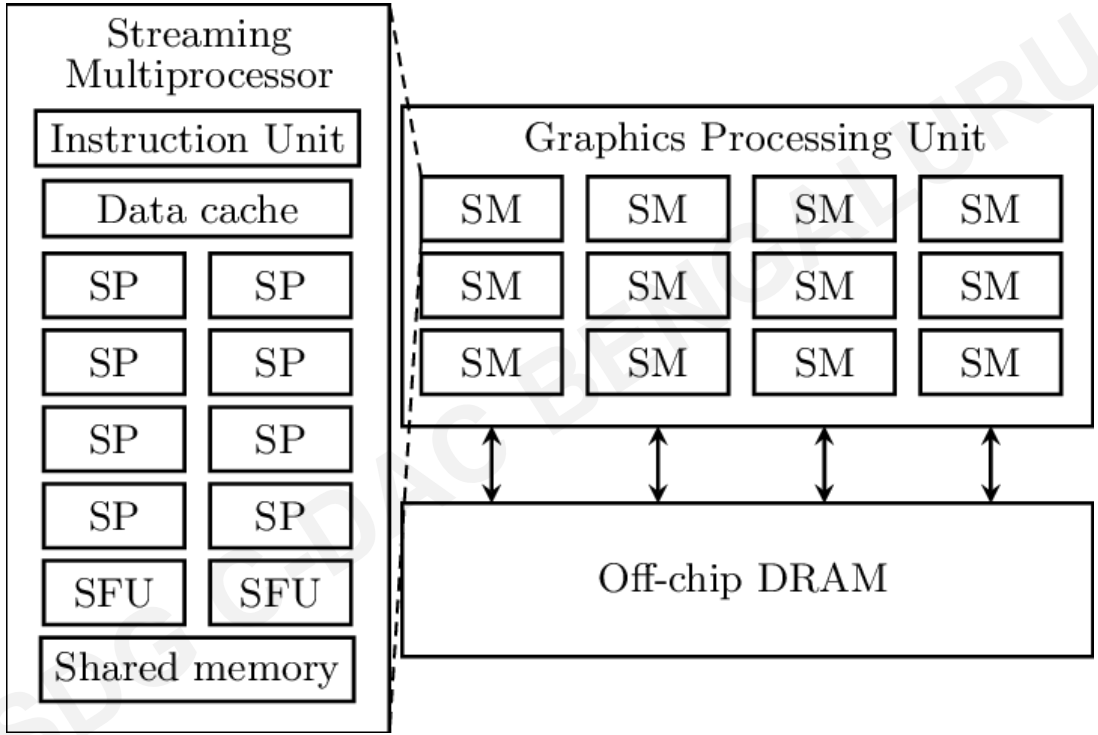


Figure: NVIDIA GPGPU Architecture

NVIDIA GPGPU Architecture

SM (Streaming Multiprocessor):

- It executes multiple threads in parallel
- It uses Single Instruction, Multiple Threads (SIMT) model.
- It contains multiple SPs, SFUs, register files, shared memory, etc.
- It can execute several warps (groups of 32 threads) concurrently.

SSDG C-DAC BENGALURU

NVIDIA GPGPU Architecture

SP (Streaming Processor):

- It is a scalar processor within an SM that executes floating-point or integer operations for a single thread.
- It is also known as a CUDA core.
- Multiple SPs form an SM.
- Analogous to an ALU in CPUs.

SSDG C-DAC BENGALURU

NVIDIA GPGPU Architecture

SFU (Special Function Unit):

- A SFU is a unit in an SM designed to perform transcendental operations such as sine, cosine, reciprocal, square root, etc.

- Optimized for non-ALU math functions.

- SFUs typically have higher latency than SPs.

SSDG C-DAC BENGALURU

NVIDIA GPGPU Architecture

Shared Memory:

- A low-latency, on-chip memory shared among threads of the same thread block within an SM.

- Useful for data reuse and inter-thread communication.

- Faster than global memory but limited in size (~48–100 KB per SM).

SSDG C-DAC BENGALURU

NVIDIA GPGPU Architecture

Data Cache:

- An on-chip memory used to cache global memory accesses to improve memory bandwidth and reduce latency.

Instruction Unit:

- A unit in the SM that fetches, decodes, and dispatches instructions to execution units like SPs and SFUs.

SSDG C-DAC BENGALURU

NVIDIA GPGPU Architecture

Off-Chip DRAM:

- It is the main memory of the GPU.
- It is also known as global memory.
- typically large but high-latency compared to shared memory or cache.
- Accessible by all threads.
- Requires explicit memory copy from host to device.

SSEG C-DAC BENGALURU

NVIDIA GPGPU Architecture

Texture:

- Hardware block that handles texture operations.

Texture Processing Cluster:

- A TPC consists of one or two SMs and a texture unit.
- It functions as independent processing unit.

SSDG C-DAC BENGALURU

NVIDIA GPGPU Architecture

- Tesla GPUs are designed specifically for high-performance computing (HPC).

- NVIDIA's latest GPU architecture is Blackwell, which powers the GeForce RTX 50 series.

- it is often better to use single-precision floats on GPUs because faster and uses less memory.

SSDG C-DAC BENGALURU

CUDA Environment Setup

- Dependencies
- Compile and Run CUDA Code

SSDG C-DAC BENGALURU

CUDA Environment Setup

- CUDA development requires a CUDA-enabled NVIDIA GPU with at least 256 MB of graphics memory.

cmd> lspcu | grep -i nvidia

- Latest NVIDIA drivers must be installed to ensure compatibility with CUDA.

cmd> nvidia-smi

SSDG C-DAC BENGALURU

CUDA Environment Setup

- Must have CUDA Toolkit provides the compiler (nvcc), libraries, and tools needed to build CUDA applications.

cmd> nvcc --version

- A compatible host compiler is required: GCC on Linux.

cmd> gcc --version

- CUDA development requires two compilers: one for the CPU (host) code and one for the GPU (device) code.

CSEG C-DAC BENGALURU

CUDA Environment Setup

- To Compile CUDA code:

cmd> nvcc demo.cu -o demo

- To Run CUDA code:

cmd> ./demo

SSDG C-DAC BENGALURU

CUDA Programming

- CUDA Execution Hierarchy
- “Hello World!”
- CUDA Constructs

SSDG C-DAC BENGALURU

CUDA execution hierarchy

Thread :

- Smallest unit of execution in CUDA.
- Executes one instance of the kernel.
- Has private registers and local memory.
- Identified by threadIdx.
- **NVPTX :** NVIDIA Virtual Parallel Thread Execution

SSDG C-DAC BENGALURU

CUDA Programming

Warp :

- A group of 32 threads.

- The basic scheduling unit on the GPU.

- All threads in a warp execute the same instruction, but on different data.

SSDG C-DAC BENGALURU

CUDA Programming

Warp :

- Occupancy : Ratio of active warps to maximum possible warps

- warp divergence : Threads in the same warp execute different paths serially

SSDG C-DAC BENGALURU

CUDA Programming

Block :

- A group of warps
- Threads in a block can share data via shared memory.
- Identified by blockIdx and defined by blockDim.
- 1024 maximum number of threads per block on most CUDA-capable GPUs

SSDG CDAC BENGALURU

CUDA Programming

Grid :

- A collection of blocks that execute the same kernel.
- All blocks execute independently.
- Specified during kernel launch using

  <<<numBlocks, threadsPerBlock>>>

SSDG C-DAC BENGALURU

CUDA Programming

Kernel :

- A GPU function (defined with __global__) launched by the host.
- Kernel can be launcheds using

  <<<numBlocks, threadsPerBlock>>>

- If a kernel is launched with more threads than the GPU supports, the kernel fails to launch.

NOTE: Grpahics Computation Load is known as Shader.

SSDG C-DAC BENGALURU

CUDA Programming

- **CUDA execution hierarchy** :

Kernel > Grid > Block > Warp > Thread

SSDG C-DAC BENGALURU

CUDA Programming : “Hello World”

```
#include <stdio.h>
#include <cuda_runtime.h>
__global__ void hello() {
    printf("Hello from GPU thread %d!\n", threadIdx.x);
}
int main() {
    hello<<<1, 5>>>();
    cudaDeviceSynchronize();
    return 0;
}
```

SSDG C-DAC BENGALURU

CUDA Programming : CUDA Constructs

- **<<<blocks, threads>>>** :

Kernel launch syntax — specifies number of blocks and threads per block.

- **threadIdx.x** :

Built-in variable: thread's index within its block (x-dimension).

SSDG C-DAC BENGALURU

CUDA Programming : CUDA Constructs

- **blockIdx.x** :

Built-in variable: block's index within the grid (x-dimension).

- **blockDim.x** :

Built-in variable: number of threads per block (x-dimension).

SSDG C-DAC BENGALURU

CUDA Programming : CUDA Constructs

CUDA Memory and Synchronization APIs

- **cudaDeviceSynchronize()** :

Host waits for all prior GPU tasks to finish — ensures result completion.

- **cudaMemcpy()** :

Copies data between host and device memory.

SSDG C-DAC BENGALURU